AbstractCharacterVisitor

+ visit(Picker, Player): boolean
+ visit(Shaman, Player): boolean
+ visit(Builder): String
+ visit(Hunter): String
+ visit(Inventor): String
+ visit(Painter): String
+ visit(Painter, Player): boolean
+ visit(Shaman): String
+ visit(Picker): String
+ visit(Inventor, Player): boolean
+ visit(Builder, Player): boolean
+ visit(Hunter, Player): boolean

AbstractGuiVisitor

+ visit(Shaman): VBox
+ visit(Picker): VBox
+ visit(Builder): VBox
+ visit(Inventor): VBox
+ visit(Painter): VBox
+ visit(Hunter): VBox

AbstractSetAndIconVisitor

+ visit(SameIconBuilding, Player, String): void
+ visit(NoMalusBuilding, Player, String): void
+ visit(SetBonus, Player, String): void
+ visit(BonusPPBuilding, Player, String): void
+ visit(BonusFood, Player, String): void
+ visit(BonusStarBuilding, Player, String): void
+ visit(AddCard, Player, String): void
+ visit(DiscountBuilding, Player, String): void
+ visit(DoubleBonusBuilding, Player, String): void
+ visit(MultiplicationBuilding, Player, String): void
+ visit(MultiplierPPBuilderBuilding, Player, String): void

<<interface>>
ActivationVisitor

+ visit(SetBonus, Player): void
+ visit(SameIconBuilding, Player): void
+ visit(BonusPPBuilding, Player): void
+ visit(BonusStarBuilding, Player): void
+ visit(MultiplicationBuilding, Player): void
+ visit(DiscountBuilding, Player): void
+ visit(MultiplierPPBuilderBuilding, Player): void
+ visit(NoMalusBuilding, Player): void
+ visit(AddCard, Player): void
+ visit(BonusFood, Player): void
+ visit(DoubleBonusBuilding, Player): void

AddAndCountCharacter

+ visit(Shaman, Player): boolean
- setBonusCalculator(Player): boolean
+ visit(Inventor, Player): boolean
+ visit(Painter, Player): boolean
+ visit(Picker, Player): boolean
+ visit(Builder, Player): boolean
+ visit(Hunter, Player): boolean

AddCard

+ acceptActivation(ActivationVisitor, Player): void
+ addArrow(): int
+ acceptAddCard(AddCardVisitorInterface, Player): int
+ print(Printer): String[]

AddCardAbstractVisitor

+ visit(BonusPPBuilding, Player): int
+ visit(SameIconBuilding, Player): int
+ visit(DoubleBonusBuilding, Player): int
+ visit(MultiplierPPBuilderBuilding, Player): int
+ visit(BonusStarBuilding, Player): int
+ visit(SetBonus, Player): int
+ visit(MultiplicationBuilding, Player): int
+ visit(AddCard, Player): int
+ visit(BonusFood, Player): int
+ visit(DiscountBuilding, Player): int
+ visit(NoMalusBuilding, Player): int

AddCardException

AddCardTest

~ acceptAddCardTest(): void
+ addArrowTest(): void
+ acceptActivationTest(): void
+ printTest(): void

AddCardVisitor

+ visit(AddCard, Player): int

<<interface>>
AddCardVisitorInterface

+ visit(AddCard, Player): int
+ visit(BonusPPBuilding, Player): int
+ visit(MultiplicationBuilding, Player): int
+ visit(SameIconBuilding, Player): int
+ visit(MultiplierPPBuilderBuilding, Player): int
+ visit(DiscountBuilding, Player): int
+ visit(NoMalusBuilding, Player): int
+ visit(SetBonus, Player): int
+ visit(BonusStarBuilding, Player): int
+ visit(DoubleBonusBuilding, Player): int
+ visit(BonusFood, Player): int

AddCardVisitorTest

+ addCardTest(): void

AddedFoodException

App

+ main(String[]): void

AppTest

+ suite(): Test
+ testApp(): void

AvailableColorsCommand

+ execute(VirtualClientInterface, ServerController): void

AvailableColorsCommand

+ execute(ClientController, TUI): void

Board

- buildingsEra1: ArrayList<Building>
- lowerCardsRow: ArrayList<Card>
- buildingsEra3: ArrayList<Building>
- upperBuildingRow: ArrayList<Card>
- players: ArrayList<Player>
- path: ArrayList<OfferCard>
- buildingsEra2: ArrayList<Building>
- serialVersionUID: long
- deck: ArrayList<Card>
- era: int
- upperCardRow: ArrayList<Card>
- turnOrderCard: TurnOrderCard
- lowerBuildingRow: ArrayList<Card>

+ getBuildingsEra2(): ArrayList<Building>
+ initializeBoard(int): void
+ getLowerBuildingRow(): ArrayList<Card>
+ shiftBuildingUpToDown(): void
+ getUpperBuildingRow(): ArrayList<Card>
+ checkUpperEvent(): ArrayList<Event>
+ shiftUpToDown(): void
+ initializeTurnOrderCard(): void
+ getLowerCardsRow(): ArrayList<Card>
+ getDeck(): ArrayList<Card>
+ initializeDeck(): void
+ getEra(): int
+ obtainPath(ArrayList<Player>): ArrayList<OfferCard>
+ getPlayers(): ArrayList<Player>
+ refillCards(): boolean
+ checkEvent(): ArrayList<Event>
+ getBuildingsEra1(): ArrayList<Building>
+ buildingPerPlayers(int): void
+ setEra(int): void
+ bringBackToTOC(Player): int
+ getTurnOrderCard(): TurnOrderCard
+ refillUpperBuildingByEra(): void
+ setCardOnBoard(): void
+ pickCard(boolean, boolean, int): Card
+ getUpperCardRow(): ArrayList<Card>
+ getBuildingsEra3(): ArrayList<Building>
+ giveFoodForTOC(Player, int): void
+ getPath(): ArrayList<OfferCard>
+ drawCard(): Card

BoardTest

~ b: Building
~ b3: Building
~ player2: Player
~ player5: Player
~ b2: Building
~ player1: Player
- board: Board
~ b1: Building
~ player4: Player
~ player3: Player

~ drawCardTest(): void
~ checkUpperEvent(): void
~ checkUpperNoEventsTest(): void
~ pickCardUpperCardRowTest(): void
~ InizializeBoardTest(): void
~ refillCardsShouldChangeEraWhenNewEraCardIsDrawn(): void
~ initializeDeckTest(): void
~ bringBackToTOCWhenThePlayerIsOnTOCTest(): void
~ shiftUpToDownTest(): void
~ checkEvent(): void
~ pickCardLowerCardRowTest(): void
+ initializeBoardTest(): void
~ refillUpperBuildingByEra3Test(): void
~ checkUpperEventReturnTest(): void
~ initializeTurnOrderCardTest(): void
~ bringBackToTOCWhenThePlayerIsOnOCTest(): void
~ shiftBuildingUpToDownTest(): void
~ setUp(): void
~ giveFoodForTOCTest(): void
~ refillCardsWithoutChangeEraTest(): void
~ refillUpperBuildingByEra2Test(): void
~ pickCardDownBuilding(): void
~ pickCardUpperBuilding(): void
~ drawCardShouldException(): void
~ setCardBoardTest(): void

BoardView

- TOCSlots: ArrayList<StackPane>
- label: Label
- downRow: HBox
- centerRow: HBox
- totems: VBox
- guiView: GuiView
- upperRow: HBox
- pool: ExecutorService
- OCSlots: ArrayList<StackPane>
- clientController: ClientController
- YCoordinates: Map<Integer, Integer>

+ getOCSlots(): ArrayList<StackPane>
+ getDownBuildingRow(): HBox
+ getDownCharacterRow(): HBox
+ getUpperBuildingRow(): HBox
+ getDeck(): HBox
+ showPath(): Node
+ getTOCSlots(): ArrayList<StackPane>
+ showUpperRow(): Node
+ showBoard(): void
+ showDownRow(): Node
+ getUpperCharacterRow(): HBox

BonusFood

+ acceptActivation(ActivationVisitor, Player): void
+ acceptFoodBonus(TurnOrderCardFoodBonus, Player): void
+ print(Printer): String[]
+ giveExtraFood(): int

BonusFoodAbstractVisitor

+ visit(SetBonus, Player): void
+ visit(AddCard, Player): void
+ visit(MultiplicationBuilding, Player): void
+ visit(MultiplierPPBuilderBuilding, Player): void
+ visit(BonusStarBuilding, Player): void
+ visit(BonusFood, Player): void
+ visit(NoMalusBuilding, Player): void
+ visit(DiscountBuilding, Player): void
+ visit(SameIconBuilding, Player): void
+ visit(BonusPPBuilding, Player): void
+ visit(DoubleBonusBuilding, Player): void

BonusFoodTest

- bonusFood: BonusFood
- player: Player

+ giveExtraFood_ShouldReturnOne_Test(): void
+ acceptFoodBonus_ShouldCallVisitor_Test(): void
+ setup(): void
+ acceptActivation_ShouldCallVisitor_Test(): void
+ printTest(): void

BonusFoodVisitor

+ visit(BonusFood, Player): void

BonusFoodVisitorTest

+ bonusFoodVisitorTest(): void

BonusPPBuilding

+ printCard(): void
+ print(Printer): String[]
+ acceptActivation(ActivationVisitor, Player): void

BonusPPBuildingTest

- bonusPPBuilding: BonusPPBuilding
- player: Player

+ acceptActivation_ShouldCallVisitor_Test(): void
+ printCardTest(): void
+ printTest(): void
+ setup(): void

BonusStarBuilding

+ acceptActivation(ActivationVisitor, Player): void
+ print(Printer): String[]

BonusStarBuildingTest

+ acceptActivationTest(): void
~ printTest(): void

Builder

- builderDiscount: Integer
- PP: Integer

+ print(Printer): String[]
+ getBuilderDiscount(): Integer
+ getPP(): Integer
+ getImagePath(): String
+ findBox(GuiVisitor): VBox
+ addCard(CharacterVisitor, Player): boolean

BuilderTest

~ addCardTest(): void
~ ImagePathTest(): void
~ printTest(): void

Building

- price: int
- serialVersionUID: long
- pp: int
- name: String

+ acceptDiscountEvent(DiscountVisitorInterface, Player, SustenanceEvent)
+ acceptSetBonus(SetAndIconVisitor, Player): void
+ acceptActivation(ActivationVisitor, Player): void
+ printCard(): void
+ acceptAddCard(AddCardVisitorInterface, Player): int
+ acceptShamanicEvent(ShamanicVisitorInterface, Player, ShamanicEvent
+ acceptEndGame(EndGameVisitorInterface, Player): int
+ acceptDiscountEvent(DiscountVisitorInterface, Player, HuntingEvent): vo
+ acceptSameIconBonus(SetAndIconVisitor, Player, String): void
+ getName(): String
+ getPrice(): int
+ getPP(): int
+ acceptDiscountEvent(DiscountVisitorInterface, Player, PaintingEvent): vo
+ acceptFoodBonus(TurnOrderCardFoodBonus, Player): void
+ getImagePath(): String

BuildingActivation

+ visit(DiscountBuilding, Player): void
+ visit(BonusStarBuilding, Player): void
+ visit(AddCard, Player): void
+ visit(SameIconBuilding, Player): void
+ visit(NoMalusBuilding, Player): void
+ visit(BonusPPBuilding, Player): void
+ visit(BonusFood, Player): void
+ visit(MultiplicationBuilding, Player): void
+ visit(DoubleBonusBuilding, Player): void
+ visit(SetBonus, Player): void
+ visit(MultiplierPPBuilderBuilding, Player): void

BuildingDTO

~ typeEvents: Events
~ typeIcons: Icons
~ price: int
~ setCompleted: int
~ ppBonus: Integer
~ pp: int
~ buildingName: String
~ multiplier: Integer
~ era: int
~ foodBonus: Integer

+ getBuildingDTOEra(): int
+ getFoodBonus(): Integer
+ getPPBonus(): Integer
+ getBuildingName(): String
+ getBuildingDTOPrice(): int
+ getTypeEvents(): Events
+ getPP(): int
+ getMultiplier(): Integer
+ getTypeIcons(): Icons
+ getSetCompleted(): int

BuildingFactory

+ createBonusPPBuilding(int, int): Building
+ createDoubleBonusBuilding(int, int, int): Building
+ createBuildingList(): ArrayList<Building>
+ createBonusStarsBuilding(int, int, int): Building
+ createBonusFood(int, int, int): Building
+ createAddCard(int, int, int): Building
+ createMultiplierPPBuilderBuilding(int, int, int): Building
+ createSetBonus(int, int, int): Building
+ createSameIconBuilding(int, int, int): Building
+ createNoMalusBuilding(int, int, int): Building
+ createDiscountBuilding(int, int, int, int, int, Icons, Events): Building
+ createMultiplicationBuilding(int, int, int, String, int): Building

BuildingFactoryTest

~ BonusFoodBuilding(): void
~ AddCard(): void
~ DoubleBonusBuilding(): void
~ testDiscountBuilding(): void
~ SetBonusBuilding(): void
~ testNoMalusBuilding(): void
~ SameIconBuilding(): void
~ testCreateBonusPPBuilding(): void
~ BonusStarsBuilding(): void

<<interface>>
BuildingInterface

+ acceptEndGame(EndGameVisitorInterface, Player): int
+ acceptSetBonus(SetAndIconVisitor, Player): void
+ acceptActivation(ActivationVisitor, Player): void
+ acceptDiscountEvent(DiscountVisitorInterface, Player, PaintingEvent): vo
+ acceptShamanicEvent(ShamanicVisitorInterface, Player, ShamanicEvent
+ acceptAddCard(AddCardVisitorInterface, Player): int
+ acceptDiscountEvent(DiscountVisitorInterface, Player, HuntingEvent): vo
+ acceptFoodBonus(TurnOrderCardFoodBonus, Player): void
+ acceptSameIconBonus(SetAndIconVisitor, Player, String): void
+ acceptDiscountEvent(DiscountVisitorInterface, Player, SustenanceEvent)

<<enumeration>>
BuildingNames

+ DiscountBuilding:
+ AddCard:
+ MultiplicationBuilding:
+ SameIconBuilding:
+ MultiplierPPBuilderBuilding:
+ SetBonus:
+ BonusStarBuilding:
+ BonusPPBuilding:
+ DoubleBonusBuilding:
+ NoMalusBuilding:
+ BonusFood:

+ values(): BuildingNames[]
+ valueOf(String): BuildingNames

BuildingsByEraDTO

~ era2: ArrayList<BuildingDTO>
~ era1: ArrayList<BuildingDTO>
~ era3: ArrayList<BuildingDTO>

+ getEra1(): ArrayList<BuildingDTO>
+ getEra3(): ArrayList<BuildingDTO>
+ getEra2(): ArrayList<BuildingDTO>

Card

- era: int
- cardType: String
- isPickable: boolean
- serialVersionUID: long

+ getIsPickable(): boolean
+ printCard(): void
+ getImagePath(): String
+ getCardType(): String
+ getEra(): int
+ print(Printer): String[]

CardDTO

<<enumeration>>
CardType

+ CHARACTER:
+ EVENT:
+ BUILDING:

+ values(): CardType[]
+ valueOf(String): CardType

Character

- numPlayers: int
- characterType: String

+ getNumPlayers(): int
+ getCharacterType(): String
+ addCard(CharacterVisitor, Player): boolean
+ print(Printer): String[]
+ isInventorAndGetIcon(CharacterVisitor): String

CharacterDTO

- ShamanStars: Integer
- characterType: CharacterType
- NumPlayers: int
- InventorIcon: String
- BuilderDiscount: Integer
- HunterSymbol: boolean
- BuilderPP: Integer

+ getHunterSymbol(): boolean
+ getBuilderDiscount(): Integer
+ getNumPlayers(): int
+ getInventorIcon(): String
+ getCharacterType(): String
+ getShamanStars(): Integer
+ getBuilderPP(): Integer

<<interface>>
CharacterInterface

+ findBox(GuiVisitor): VBox
+ addCard(CharacterVisitor, Player): boolean

<<enumeration>>
CharacterType

+ SHAMAN:
+ INVENTOR:
+ BUILDER:
+ HUNTER:
+ PAINTER:
+ PICKER:

+ values(): CharacterType[]
+ valueOf(String): CharacterType

<<interface>>
CharacterVisitor

+ visit(Builder): String
+ visit(Shaman): String
+ visit(Painter, Player): boolean
+ visit(Inventor, Player): boolean
+ visit(Inventor): String
+ visit(Picker, Player): boolean
+ visit(Hunter, Player): boolean
+ visit(Hunter): String
+ visit(Picker): String
+ visit(Painter): String
+ visit(Shaman, Player): boolean
+ visit(Builder, Player): boolean

<<interface>>
ClientCommand

+ execute(Socket, ClientController): void

ClientController

- serverConnection: ServerConnection
- tmpUsername: String
- BoardLock: Object
- pingManager: PingManagerClientSide
- localNumberRound: int
- pickingPhase: AtomicBoolean
- isRunning: AtomicBoolean
- localTotem: Totem
- serverIP: String
- scheduler: ScheduledExecutorService
- currentBoard: Board
- isGameFinished: AtomicBoolean
- tmpTotem: Totem
- localPlayerName: String
- tmpLock: Object
- leaderboardDB: List<LeaderBoardData>
- view: GraphicInterface
- playerPosition: Integer

+ showSkipError(): void
+ setTmpUsername(String): void
+ showUpdateTurn(Board, int): void
+ showEndGame(): void
+ setView(GraphicInterface): void
+ checkForLogin(): void
+ showLocalReturnToTOC(String, int): void
+ showAvailableTotems(ArrayList<Totem>): void
+ showConfirmUsername(String): void
+ leave(): void
+ showLocalUpdateEra(): void
+ foodUpdated(String, int): void
+ getPlayerByName(String): Player
+ getLocalTotem(): Totem
+ requestSkip(): void
+ showPlayerTurn(String): void
+ confirmNumPlayers(int): void
+ showGameStarted(ArrayList<Player>, Board): void
+ close(): void
+ showConfirmTotem(Totem): void
+ requestLocalMoveTotem(int): void
+ showUpdateFirstPlayer(): void
+ showPickedCardError(): void
+ setCurrentBoard(Board): void
+ getCurrentBoard(): Board
+ setServerConnection(boolean): void
+ showUpdateForEvents(Event): void
+ showSkipMessage(): void
+ updatePlayerPP(String, int): void
+ showTotemMovedError(): void
+ handleEndGameNormally(String, List<PlayerScore>): void
+ getLocalPlayerName(): String
+ showNumPlayers(int): void
+ showTotemMoved(String, int): void
+ getConnection(): ServerConnection
+ requestLocalAvailableTotems(): void
+ setTmpTotem(Totem): void
+ showError(String): void
+ showPickedCard(String, boolean, boolean, int, int): void
+ showUsernameError(): void
+ updateLeaderboardFromDB(int, List<LeaderBoardData>): void
+ refuseOfConnection(): void
+ numPlayersChosenError(): void
+ setNumPlayers(int): void
+ handleForcedEndGame(): void
+ getLocalPlayer(): Player
+ requestLocalPickCard(boolean, boolean, int): void
+ showTotemChoiceError(): void
+ setConnection(ServerConnection): void

ClientDisconnectedException

ClientHandler

- serverController: ServerController
- reader: ObjectInputStream
+ terminationSignal: Boolean
~ client: VirtualClientInterface
- socket: Socket
- pool: ExecutorService

+ run(): void
+ close(): void
- cleanup(): void

<<interface>>
ClientInterface

+ login(String, Totem): void
+ setNumPlayers(int): void
+ leave(VirtualClientInterface): void
+ moveTotem(String, int): void
+ pickCard(String, boolean, boolean, int): void
+ setTotemPosition(int): void

ClientMain

- tui: TUI
~ yellowStyle: AttributedStyle
~ yellowBoldStyle: AttributedStyle
- clientController: ClientController
- isTerminated: AtomicBoolean
- terminal: Terminal

+ terminateClient(): void
+ main(String[]): void

ClientRMI

- serverName: String
- ipAddress: String
- server: VirtualServer
- port: int
- clientController: ClientController
- username: String

+ showTotemMovedError(String): void
+ updateNextTurn(Board, int): void
+ requestMoveTotem(String, int): void
+ numPlayersChosenError(String): void
+ showChosenNumPlayers(int): void
+ showPickedCard(String, boolean, boolean, int, int): void
+ showAvailableTotems(ArrayList<Totem>): void
+ requestAvailableTotems(): void
+ showUpdateFirstPlayer(): void
+ requestPickCard(String, boolean, boolean, int, boolean): void
+ showSkipError(String): void
+ showLeaderboardFromDB(int, List<LeaderBoardData>): void
+ showUsernameError(String): void
+ showConfirmUsername(String): void
+ login(String, Totem): void
+ showForcedEndGame(): void
+ showUpdateForEvent(Event): void
+ showUpdatePlayerPP(String, int): void
+ setClientController(ClientController): void
+ showGameStarted(ArrayList<Player>, Board): void
+ showLocalUpdateEra(int): void
+ showConfirmTotem(Totem): void
+ refuseConnection(): void
+ run(): void
+ showPlayerTurn(String): void
+ showTotemMoved(String, int): void
+ close(): void
+ ping(): void
+ requestSetNumPlayers(int): void
+ showTotemChoiceError(String): void
+ leave(): void
- ipAddressChoice(String): String
+ showLocalReturnToTOC(String, int): void
+ showUpdatePlayerFood(String, int): void
+ showPickCardError(String): void
+ showEndGameSuccessfully(String, List<PlayerScore>): void

CloseConnectionCommand

+ execute(Socket, ClientController): void

CommandFactoryClientSide

- commands: Map<String, Function<Object, ClientCommand>>

+ getCommand(MessageFromServer): ClientCommand

CommandFactoryServer

- commands: Map<String, Function<Object, ServerCommand>>

+ getCommand(MessageFromClient): ServerCommand

CommandFactoryTUI

- commandsMap: Map<String, Function<String, CommandTUI>>
- separator: char
- noSepCommandsMap: Map<String, Function<String, CommandTUI>>

+ createNoSepCommandsMap(): void
+ getCommand(String): CommandTUI
+ createCommandsMap(): void

<<interface>>
CommandTUI

+ execute(ClientController, TUI): void

ConcreteBuildingActivation

+ visit(SetBonus, Player): void
+ visit(BonusStarBuilding, Player): void
+ visit(AddCard, Player): void
+ visit(SameIconBuilding, Player): void

ConcreteFactoryEra

- era: int

+ createInventor(int, int, String): Character
+ createHunter(int, int, boolean): Character
+ createEventList(): ArrayList<Event>
+ createCharacterList(): ArrayList<Character>
+ createPainter(int, int): Character
+ createShamanicEvent(int, int, int): Event
+ createHuntingEvent(int, int): Event
+ createPaintingEvent(int, int, int, int): Event
+ createPicker(int, int): Character
+ createSustenanceEvent(int, int): Event
+ createShaman(int, int, int): Character
+ createBuilder(int, int, int, int): Character

ConcreteFactoryEraTest

~ testCreateInventor(): void

ConcreteSetAndIconVisitor

+ visit(SetBonus, Player, String): void
+ visit(SameIconBuilding, Player, String): void

ConcreteSetAndIconVisitorTest

+ concreteSetAndIconVisitorTest(): void

ConfirmNumPlayers

~ num: int

+ execute(Socket, ClientController): void

ConfirmTotemCommand

- totem: Totem

+ execute(Socket, ClientController): void

ConfirmUsernameCommand

- username: String

+ execute(Socket, ClientController): void

ConnectionSelectionCommand

- choice: int

+ execute(ClientController, TUI): void

<<record>>
DBData

- position: int
- leaderboard: List<LeaderBoardData>

+ position(): int
+ leaderboard(): List<LeaderBoardData>

DBLeaderBoardCommand

~ leaderboard: List<LeaderBoardData>
~ PlayerPositionInDB: int

+ execute(Socket, ClientController): void

DBTest

+ leaderBoardTest(): void
+ checkLeaderBoardPositionTest(): void
+ DBTest(): void
+ addScoreTest(): void

DatabaseManager

- properties: Properties

+ createTable(): void
+ getConnection(): Connection

Deck

- deck: ArrayList<Card>
- serialVersionUID: long

+ createDeck(int): ArrayList<Card>
+ getDeck(): ArrayList<Card>

DeckTest

+ createDeckTest3(): void
+ createDeckTest5(): void
+ createDeckTest2(): void
+ createDeckTest4(): void

DiscountAbstractVisitor

+ visit(BonusFood, Player, SustenanceEvent): int
+ visit(MultiplicationBuilding, Player, PaintingEvent): void
+ visit(SameIconBuilding, Player, HuntingEvent): void
+ visit(MultiplierPPBuilderBuilding, Player, PaintingEvent): void
+ visit(MultiplierPPBuilderBuilding, Player, SustenanceEvent): int
+ visit(SameIconBuilding, Player, PaintingEvent): void
+ visit(AddCard, Player, HuntingEvent): void
+ visit(NoMalusBuilding, Player, SustenanceEvent): int
+ visit(SameIconBuilding, Player, SustenanceEvent): int
+ visit(SetBonus, Player, HuntingEvent): void
+ visit(MultiplierPPBuilderBuilding, Player, HuntingEvent): void
+ visit(BonusPPBuilding, Player, HuntingEvent): void
+ visit(BonusStarBuilding, Player, PaintingEvent): void
+ visit(DiscountBuilding, Player, SustenanceEvent): int
+ visit(DiscountBuilding, Player, PaintingEvent): void
+ visit(DiscountBuilding, Player, HuntingEvent): void
+ visit(BonusFood, Player, HuntingEvent): void
+ visit(SetBonus, Player, SustenanceEvent): int
+ visit(BonusFood, Player, PaintingEvent): void
+ visit(BonusPPBuilding, Player, PaintingEvent): void
+ visit(BonusStarBuilding, Player, SustenanceEvent): int
+ visit(NoMalusBuilding, Player, PaintingEvent): void
+ visit(NoMalusBuilding, Player, HuntingEvent): void
+ visit(MultiplicationBuilding, Player, HuntingEvent): void
+ visit(BonusStarBuilding, Player, HuntingEvent): void
+ visit(AddCard, Player, PaintingEvent): void
+ visit(BonusPPBuilding, Player, SustenanceEvent): int
+ visit(DoubleBonusBuilding, Player, PaintingEvent): void
+ visit(DoubleBonusBuilding, Player, SustenanceEvent): int
+ visit(SetBonus, Player, PaintingEvent): void
+ visit(DoubleBonusBuilding, Player, HuntingEvent): void
+ visit(AddCard, Player, SustenanceEvent): int
+ visit(MultiplicationBuilding, Player, SustenanceEvent): int

DiscountBuilding

- foodBonus: int
- ppBonus: int
- typeEvents: Events
- typeIcons: Icons

+ getFoodBonusForSustenance(Player): int
+ getTypeEvents(): Events
+ acceptDiscountEvent(DiscountVisitorInterface, Player, HuntingEvent): v
+ print(Printer): String[]
+ getPpBonus(Player): int
+ acceptActivation(ActivationVisitor, Player): void
+ getTypeIcons(): Icons
+ getImagePath(): String
+ acceptDiscountEvent(DiscountVisitorInterface, Player, PaintingEvent):
+ acceptDiscountEvent(DiscountVisitorInterface, Player, SustenanceEve
+ getPpBonus(): int
+ getFoodBonusForHunting(Player): int
+ getFoodBonus(): int
+ getFoodBonusForPainting(Player): int

DiscountBuildingTest

~ factory: ConcreteFactoryEra
~ characters: ArrayList<Character>
~ buildingFactory: BuildingFactory
~ p3: Player
~ buildings: ArrayList<Building>
~ p2: Player
~ players: ArrayList<Player>
~ events: ArrayList<Event>
~ p1: Player

+ getPPBonusTest(): void
+ discountBuildingTest1(): void
~ acceptActivationForHuntingEvent_Test(): void
~ discountBuildingForPaintingEvent_Test(): void
+ imagePathTest(): void
+ discountBuildingTest(): void
+ printCardTest(): void

DiscountVisitor

+ visit(DiscountBuilding, Player, PaintingEvent): void
+ visit(DiscountBuilding, Player, SustenanceEvent): int
+ visit(DiscountBuilding, Player, HuntingEvent): void

<<interface>>
DiscountVisitorInterface

+ visit(MultiplicationBuilding, Player, PaintingEvent): void
+ visit(SetBonus, Player, PaintingEvent): void
+ visit(DiscountBuilding, Player, SustenanceEvent): int
+ visit(DoubleBonusBuilding, Player, PaintingEvent): void
+ visit(SameIconBuilding, Player, PaintingEvent): void
+ visit(DoubleBonusBuilding, Player, SustenanceEvent): int
+ visit(SetBonus, Player, HuntingEvent): void
+ visit(BonusPPBuilding, Player, PaintingEvent): void
+ visit(SameIconBuilding, Player, SustenanceEvent): int
+ visit(NoMalusBuilding, Player, SustenanceEvent): int
+ visit(BonusStarBuilding, Player, PaintingEvent): void
+ visit(NoMalusBuilding, Player, PaintingEvent): void
+ visit(BonusPPBuilding, Player, HuntingEvent): void
+ visit(MultiplierPPBuilderBuilding, Player, HuntingEvent): void
+ visit(AddCard, Player, PaintingEvent): void
+ visit(MultiplicationBuilding, Player, HuntingEvent): void
+ visit(NoMalusBuilding, Player, HuntingEvent): void
+ visit(BonusStarBuilding, Player, SustenanceEvent): int
+ visit(AddCard, Player, SustenanceEvent): int
+ visit(SetBonus, Player, SustenanceEvent): int
+ visit(MultiplierPPBuilderBuilding, Player, SustenanceEvent): int
+ visit(AddCard, Player, HuntingEvent): void
+ visit(MultiplicationBuilding, Player, SustenanceEvent): int
+ visit(BonusStarBuilding, Player, HuntingEvent): void
+ visit(BonusFood, Player, HuntingEvent): void
+ visit(DiscountBuilding, Player, HuntingEvent): void
+ visit(DiscountBuilding, Player, PaintingEvent): void
+ visit(DoubleBonusBuilding, Player, HuntingEvent): void
+ visit(SameIconBuilding, Player, HuntingEvent): void
+ visit(BonusPPBuilding, Player, SustenanceEvent): int
+ visit(BonusFood, Player, SustenanceEvent): int
+ visit(BonusFood, Player, PaintingEvent): void
+ visit(MultiplierPPBuilderBuilding, Player, PaintingEvent): void

DiscountVisitorTest

+ DiscountVisitorForPainterTest(): void
+ DiscountVisitorForSustenanceTest(): void
+ DiscountVisitorForHuntingTest(): void

DoubleBonusBuilding

+ acceptActivation(ActivationVisitor, Player): void
+ acceptShamanicEvent(ShamanicVisitorInterface, Player, ShamanicEvent
+ print(Printer): String[]
+ giveDouble(): int

DoubleBonusBuildingTest

- player: Player
- event: ShamanicEvent
- building: DoubleBonusBuilding

+ acceptActivation_ShouldCallVisitor_Test(): void
+ acceptShamanicEvent_ShouldCallVisitor_Test(): void
+ printTest(): void
+ giveDouble_ShouldReturnTwo_Test(): void
+ setup(): void

EndGameAbstractVisitor

+ visit(DoubleBonusBuilding, Player): int
+ visit(BonusFood, Player): int
+ visit(SetBonus, Player): int
+ visit(MultiplierPPBuilderBuilding, Player): int
+ visit(BonusStarBuilding, Player): int
+ visit(DiscountBuilding, Player): int
+ visit(NoMalusBuilding, Player): int
+ visit(BonusPPBuilding, Player): int
+ visit(SameIconBuilding, Player): int
+ visit(MultiplicationBuilding, Player): int
+ visit(AddCard, Player): int

EndGameBuilding

typeIcons: String

+ getTypeIcons(): String
+ countPP(Player): int

EndGameCommand

~ leaderboard: List<PlayerScore>
~ winner: String

+ execute(Socket, ClientController): void

<<record>>
EndGameData

- winner: String
- leaderboard: List<PlayerScore>

+ winner(): String
+ leaderboard(): List<PlayerScore>

EndGameVisitor

+ visit(MultiplicationBuilding, Player): int
+ visit(MultiplierPPBuilderBuilding, Player): int

<<interface>>
EndGameVisitorInterface

+ visit(NoMalusBuilding, Player): int
+ visit(BonusStarBuilding, Player): int
+ visit(AddCard, Player): int
+ visit(MultiplicationBuilding, Player): int
+ visit(BonusPPBuilding, Player): int
+ visit(MultiplierPPBuilderBuilding, Player): int
+ visit(BonusFood, Player): int
+ visit(DiscountBuilding, Player): int
+ visit(SameIconBuilding, Player): int
+ visit(SetBonus, Player): int
+ visit(DoubleBonusBuilding, Player): int

EraDTO

- era: int
- characters: ArrayList<CharacterDTO>
- events: ArrayList<EventDTO>

+ getEventsArray(): ArrayList<EventDTO>
+ getEra(): int
+ getCharactersArray(): ArrayList<CharacterDTO>

EraDTOTest

~ zeroShouldReturnIllegalArgumentException(): void
~ eventsShouldBeInList(): void
~ threeShouldReturnEra(): void
~ characterShouldBeInList(): void

Event

- serialVersionUID: long
- eventName: String

+ printCard(): void
+ resolveEvent(ArrayList<Player>): void
+ getImagePath(): String
+ getEventName(): String

EventAbstractTest

~ printCardTest(): void
~ getImagePathSustenanceEvent(): void

EventDTO

- PaEveNumMinPainters: Integer
- ShEvePrizePoints: Integer
- PaEveMultiplierPP: Integer
- SuEvePointsLossMultiplier: Integer
- ShEvePenPoints: Integer
- eventName: String
- FinalEvent: Boolean
- HuEvePP: Integer
- PaEvePointsLoss: Integer

+ getPaEveNumMinPainters(): Integer
+ getPaEvePointsLoss(): Integer
+ getSuEvePointsLossMultiplier(): Integer
+ isBetween(Integer, Integer, Integer): boolean
+ getShEvePenPoints(): Integer
+ getShEvePrizePoints(): Integer
+ getPaEveMultiplierPP(): Integer
+ isFinalEvent(): Boolean
+ getHuEvePP(): Integer
+ getEventName(): String

EventDTOTest

<<enumeration>>
EventName

+ PAINTING_EVENT:
+ SHAMANIC_EVENT:
+ HUNTING_EVENT:
+ SUSTENANCE_EVENT:

+ valueOf(String): EventName
+ values(): EventName[]

<<enumeration>>
Events

+ HUNTEREVENT:
+ PAINTINGEVENT:
+ SUSTENANCEEVENT:

+ values(): Events[]
+ valueOf(String): Events

<<interface>>
Factory

+ createShamanicEvent(int, int, int): Event
+ createInventor(int, int, String): Character
+ createHuntingEvent(int, int): Event
+ createHunter(int, int, boolean): Character
+ createBuilder(int, int, int, int): Character
+ createShaman(int, int, int): Character
+ createPicker(int, int): Character
+ createPaintingEvent(int, int, int, int): Event
+ createSustenanceEvent(int, int): Event
+ createPainter(int, int): Character

GameManager

- currentPlayer: Player
- pickingQueue: List<PendingPick>
+ numPlayersLock: Object
- board: Board
- players: ArrayList<Player>
- isGameStarted: AtomicBoolean
- numPlayers: int
+ playersLock: Object
- positioningPhase: AtomicBoolean
- notifier: Notifier
- round: int
- pickingPhase: AtomicBoolean

+ endGame(): void
+ isDirectionPickable(): boolean
+ initializeFood(): void
+ getNumPlayers(): int
+ setBoard(Board): void
+ getPlayerByName(String): Player
+ setPickingPhaseForTest(boolean): void
+ setNumPlayers(int): void
+ setNotifier(Notifier): void
+ getCurrentPlayer(): Player
+ addPlayer(Player): void
+ resolvePick(String, boolean, boolean, int): void
+ buyBuilding(Player, boolean, int): Building
+ getPlayers(): ArrayList<Player>
+ endRound(): void
- checkAndReturnToTOC(Player): void
~ availableBuilding(ArrayList<Card>): boolean
+ nextRound(): void
+ resolveEvents(ArrayList<Event>): void
+ executeNextPosition(): void
+ resolvePosition(String, int): void
+ getPickingQueue(): List<PendingPick>
+ pickingPhase(): void
+ isPickingPhase(): boolean
+ executeNextPick(): void
~ availableCharacters(ArrayList<Card>): boolean
+ getBoard(): Board
+ skipPick(String): void
+ getRound(): int
+ positionPhase(): void
+ takeCharacter(Player, boolean, int): boolean
+ startGame(): void

GameManagerTest

~ player1: Player
~ player3: Player
~ player4: Player
~ player2: Player
~ player5: Player

+ inizializeFood_For3PlayersTest(): void
+ resolvePickTest(): void
+ endGameTest(): void
+ isDirectionPickable_MixedContent_Test(): void
+ isDirectionPickable_NoCardsAvailable_Test(): void
+ getPlayerByNameTest(): void
+ skipPickSuccessTest(): void
+ initializeFood_For5PlayersTest(): void
+ executeNextPickisEmptyTest(): void
+ PurchaseBuildingFailureTest(): void
+ nextRound_ShouldEndGameAtRound11(): void
+ resolveEvents_ShouldPlaceSustenanceLast(): void
+ executeNextPickNotEmptyTest(): void
+ nextRoundIncrementTest(): void
+ nextRoundTest(): void
+ setNumPlayersTest(): void
+ pickingPhaseTest(): void
+ BuyBuildingTest(): void
+ getPlayerByNameFailure_Test(): void
+ isDirectionPickable_CharactersAvailableTest(): void
+ executeNextPosition_WhenOrderIsEmptyTest(): void
+ PurchaseBuildingWithDiscountTest(): void
+ executeNextPosition_WhenOrderIsNotEmptyTest(): void
+ skipPickNotSkippableTest(): void
+ resolvePickFailureTest(): void
+ TakeCharacterTest(): void
+ endRound_ContinuesIfDeckHasCards(): void
+ resolvePositionTest(): void
+ addPlayerSuccessTest(): void
+ endRound_EndsGameIfDeckIsEmpty(): void
+ addPlayerLimitReachedTest(): void
+ pickingPhase_ShouldGiveFood_When5Players(): void
+ isDirectionPickable_OnlyBuildingsAvailable_Test(): void
+ startGameTest(): void
+ addPlayerAfterGameStartTest(): void
+ skipPickFailureTest(): void

<<interface>>
GraphicInterface

+ showReturnToTOC(String): void
+ showError(String): void
+ showPlayerPPUpdate(String, int): void
+ openManual(): void
+ showLeaderboardFromDB(int, List<LeaderBoardData>): void
+ showErrorMessage(String): void
+ showPlayerFoodUpdate(String, int): void
+ showStartGame(): void
+ showAvailableTotems(ArrayList<Totem>): void
+ showNextRound(): void
+ showEndGameSuccessfully(String, List<PlayerScore>): void
+ close(): void
+ askNumToPlayer(): void
+ moveTotem(String, int): void
+ pickCard(String, boolean, boolean, int): void
+ showMessage(String): void
+ showCurrentPlayer(String): void
+ showLobbyMenu(): void

GuiCharacterVisitor

- boxes: Map<Class, VBox>

+ visit(Painter): VBox
+ visit(Picker): VBox
+ visit(Builder): VBox
+ visit(Shaman): VBox
+ visit(Inventor): VBox
+ visit(Hunter): VBox

GuiView

- menuView: MenuView
- highlightedPosition: int
- currentPlayerLabel: Label
- root: StackPane
- positionColumn: TableColumn<LeaderBoardData, Integer>
- boardView: BoardView
- scoreColumn: TableColumn<LeaderBoardData, Integer>
- panels: Map<String, Playerpanel>
- pool: ExecutorService
- clientController: ClientController
- dateColumn: TableColumn<LeaderBoardData, String>
- usernameColumn: TableColumn<LeaderBoardData, String>
- leaderboardTable: TableView<LeaderBoardData>
- vboxMessage: VBox
- label: Label

+ showAvailableTotems(ArrayList<Totem>): void
+ showLeaderboardFromDB(int, List<LeaderBoardData>): void
+ showPlayersBar(): Node
+ showNextRound(): void
+ showLocalPlayer(): Node
+ moveTotem(String, int): void
+ showStartGame(): void
+ showErrorMessage(String): void
+ refreshDownCards(): void
+ showError(String): void
+ pickCard(String, boolean, boolean, int): void
+ refreshDownBuildingCards(): void
+ showPlayerFoodUpdate(String, int): void
+ close(): void
+ showPlayerPPUpdate(String, int): void
+ showEndGameSuccessfully(String, List<PlayerScore>): void
+ refreshUpperCards(): void
+ showReturnToTOC(String): void
+ askNumToPlayer(): void
+ initializeLeaderboardDB(): void
+ updateReferences(): void
+ showMainStage(): void
+ showMessage(String): void
+ showLobbyMenu(): void
+ showCurrentPlayer(String): void
+ refreshUpperBuildingCards(): void

<<interface>>
GuiVisitor

+ visit(Painter): VBox
+ visit(Hunter): VBox
+ visit(Inventor): VBox
+ visit(Shaman): VBox
+ visit(Picker): VBox
+ visit(Builder): VBox

HelpCommand

+ execute(ClientController, TUI): void

Hunter

- symbol: boolean

+ getSymbol(): boolean
+ getImagePath(): String
+ findBox(GuiVisitor): VBox
+ print(Printer): String[]
+ addCard(CharacterVisitor, Player): boolean

HunterTest

~ printHunterTest(): void
~ ImagePathTest(): void

HuntingEvent

- HuEvePP: Integer

+ printCard(): void
+ resolveEvent(ArrayList<Player>): void
+ getHuEvePP(): Integer
+ print(Printer): String[]

HuntingEventTest

~ visitor: CharacterVisitor
~ p1: Player
~ p2: Player
~ p3: Player

~ testHuntingEvent(): void
~ shouldThrowExceptionForSinglePlayer(): void
+ printCardDoesNotThrowTest(): void

<<enumeration>>
Icons

+ INVENTOR:
+ HUNTER:
+ SET:
+ BUILDER:
+ PAINTER:
+ SHAMAN:
+ PICKER:

+ values(): Icons[]
+ valueOf(String): Icons

InvalidFormatCommand

+ execute(ClientController, TUI): void

Inventor

- inventorIcon: String

+ getImagePath(): String
+ isInventorAndGetIcon(CharacterVisitor): String
+ findBox(GuiVisitor): VBox
+ addCard(CharacterVisitor, Player): boolean
+ getInventorIcon(): String
+ print(Printer): String[]

<<enumeration>>
InventorIcon

+ LEATHER:
+ BOWL:
+ DOLL:
+ BREAD:
+ BOAT:
+ TREE:
+ HOOK:
+ NECKLACE:
+ FLUTE:
+ KNOT:

+ valueOf(String): InventorIcon
+ values(): InventorIcon[]

InventorIconCounter

+ visit(Inventor): String

<<interface>>
InventorInterface

+ isInventorAndGetIcon(CharacterVisitor): String

InventorTest

~ ConstructorIconTest(): void
~ ImagePathTest(): void
~ printTest(): void

LeaderBoardDAO

+ addNewPlayerScore(String, int, int): void
+ getAllTimeLeaderBoard(int): List<LeaderBoardData>
+ getPositionInLeaderBoard(int, int): int

<<record>>
LeaderBoardData

- date: String
- position: int
- score: int
- username: String

+ score(): int
+ date(): String
+ username(): String
+ position(): int

Loader

+ loadBuildingsByEra(): BuildingsByEraDTO
+ loadFile(int): EraDTO

LoaderTest

~ SearchEra(): void
~ LoadBuildingsIsRight(): void

Lobby

- tempPlayers: ArrayList<TempPlayer>
+ numPlayers: AtomicInteger
+ isNumPlayersSet: AtomicBoolean
+ serverController: LobbyManager

+ getTempPlayerByClient(VirtualClientInterface): TempPlayer
+ getTempPlayers(): ArrayList<TempPlayer>
+ setNumPlayers(Integer, VirtualClientInterface): void
+ IsNumPlayersSet(): AtomicBoolean
+ sendAvailableColors(VirtualClientInterface): void
+ getNumPlayers(): AtomicInteger
+ addTotem(Totem, VirtualClientInterface): void
+ checkMoreThenEnoughPlayers(): void
+ addUsername(String, VirtualClientInterface): void
+ getAvailableTotems(): ArrayList<Totem>
+ addClient(VirtualClientInterface): void

<<interface>>
LobbyManager

+ checkSetNumPlayers(int, VirtualClientInterface): void
+ checkTotem(Totem, VirtualClientInterface): void
+ checkUsername(String, VirtualClientInterface): void
+ connectionInitializer(VirtualClientInterface): void

LobbyTest

- lobby: Lobby
- Client: Client
- fakeServerController: FakeServerController

~ checkMoreThenEnoughPlayersTest(): void
~ getTempPlayerByClientNotFoundTest(): void
~ getAvailableTotemsInitialTest(): void
~ setUp(): void
~ addFirstClientTest(): void
~ checkMoreThenEnoughPlayersExcessTest(): void
~ addUsernameTest(): void
~ duplicateClientNotAddedTest(): void
~ getTempPlayerByClientFoundTest(): void
~ getAvailableTotemsTest(): void
~ checkMoreThenEnoughPlayersNotSetTest(): void
~ setNumPlayersTest(): void
~ sendAvailableColorsTest(): void
~ refuseConnectionWhenFullTest(): void
~ addTotemTest(): void

MainApp

- clientController: ClientController
- root: StackPane

+ main(String[]): void
+ setController(ClientController): void
+ start(Stage): void

MenuView

- pool: ExecutorService
- totemsLine: VBox
- guiView: GuiView
- connection: VBox
- login: HBox
- username: VBox
- numPlayers: VBox
- clientController: ClientController

+ chooseConnection(): VBox
+ createSchema(): void
+ chooseUsername(): VBox
+ showAvailableTotems(ArrayList<Totem>): VBox
+ getNumPlayers(): VBox

MessageFromClient

- serialVersionUID: long
- payload: T
- header: String

+ getHeader(): String
+ getPayload(): T

MessageFromServer

- header: String
- payload: T
- serialVersionUID: long

+ getHeader(): String
+ getPayload(): T

MoveTotemCommand

- totemPosition: TotemPosition

+ execute(VirtualClientInterface, ServerController): void

MoveTotemCommand

~ provvIndex: String

+ execute(ClientController, TUI): void

MovedTotemCommand

~ pos: TotemPosition

+ execute(Socket, ClientController): void

MultiplicationBuilding

- multiplier: int

+ countPP(Player): int
+ getMultiplier(): int
+ acceptActivation(ActivationVisitor, Player): void
+ getImagePath(): String
+ acceptEndGame(EndGameVisitorInterface, Player): int
+ print(Printer): String[]

MultiplicationBuildingTest

~ p: Player
~ factory: ConcreteFactoryEra
~ characters: ArrayList<Character>

~ countPPIconTest(): void
+ printTest(): void
+ ImagePathTest(): void
~ testCountPP_WithSets(): void
~ testCountPP_WithIcons(): void
+ acceptEndGame_ReturnsVisitorValue_Test(): void
+ acceptActivation_CallsVisitor_Test(): void

MultiplierPPBuilderBuilding

+ printCard(): void
+ acceptActivation(ActivationVisitor, Player): void
+ print(Printer): String[]
+ acceptEndGame(EndGameVisitorInterface, Player): int
+ countPP(Player): int

MultiplierPPBuilderBuildingTest

~ building: MultiplierPPBuilderBuilding
~ p1: Player

+ printTest(): void
+ printCardTest(): void
+ acceptEndGameTest(): void
+ countPPTest(): void
+ acceptActivationTest(): void

NextRoundCommand

- round: int
- board: Board

+ execute(Socket, ClientController): void

<<record>>
NextRoundData

- round: int
- board: Board

+ round(): int
+ board(): Board

NoMalusBuilding

+ acceptShamanicEvent(ShamanicVisitorInterface, Player, ShamanicEvent
+ print(Printer): String[]
+ noMalus(): boolean
+ acceptActivation(ActivationVisitor, Player): void

NoMalusBuildingTest

~ p: Player
~ b: NoMalusBuilding
~ v: ActivationVisitor

~ acceptShamanicEventTest(): void
~ noMalusActivationTest(): void
~ printTest(): void
~ acceptActivationAndVisitorTest(): void
~ acceptActivationShouldCallVisitor(): void
~ constructorNoMalusBuildingTest(): void

NotFoundCommand

- command: String

+ execute(Socket, ClientController): void

Notifier

- clients: ArrayList<VirtualClientInterface>
- startTermProcedure: AtomicBoolean
- outputLock: Object
- pool: ExecutorService

+ addClientToNotifier(VirtualClientInterface): void
+ nextRoundBroadcast(Board, int): void
+ gameStartedBroadcast(ArrayList<Player>, Board): void
+ ppUpdateBroadcast(Player, int): void
+ allowSkipTurn(Player): void
+ showEndGameBroadcast(Player, List<PlayerScore>): void
+ invalidBuildingPurchase(Player): void
+ newEraBroadcast(int): void
+ pickedCardBroadcast(Player, boolean, boolean, int, int): void
+ movedTotemBroadcast(Player, int): void
+ sendLeaderBoard(Player, int, ArrayList<LeaderBoardData>): void
+ foodUpdateBroadcast(Player, int): void
+ invalidCardPick(Player): void
- getClients(): List<VirtualClientInterface>
- handleDisconnect(VirtualClientInterface): void
+ resolvingEventBroadcast(Event): void
+ sendFarewell(VirtualClientInterface): void
+ invalidTotemPosition(Player): void
+ returnTotemOnTurnOrderBroadcast(Player, int): void
+ showTurnBroadcast(Player): void
+ terminationSignalBroadcast(): void

NumPlayersCommand

+ execute(Socket, ClientController): void

OfferCard

- DownArrow: int
- serialVersionUID: long
- Food: boolean
- ID: int
- OccupiedBy: Player
- IsOccupied: boolean
- UpArrow: int

+ getOccupiedBy(): Player
+ release(): void
+ getID(): int
+ isOccupied(): boolean
+ print(Printer): String[]
+ getUpArrow(): int
+ isFood(): boolean
+ getDownArrow(): int
+ setOccupiedBy(Player): void

OfferCardTest

+ releaseAnOccupiedOfferCardTest(): void
+ OfferCardConstructorTest(): void
~ printTest(): void
+ createOfferCardRight(): void
+ OfferCardOccupiedBy(): void
~ setOccupiedByWithNullParameter(): void
+ OfferCardOccupyAndReleseTest(): void

Painter

+ findBox(GuiVisitor): VBox
+ addCard(CharacterVisitor, Player): boolean
+ getImagePath(): String
+ print(Printer): String[]

PainterTest

~ painterImagePathTest(): void
~ painterPrintTest(): void
~ constructorTest1(): void
~ constructorTest(): void

PaintingEvent

- PaEveMultiplierPP: Integer
- PaEveNumMinPainters: Integer
- PaEvePointsLoss: Integer

+ getPaEvePointsLoss(): Integer
+ getPaEveNumMinPainters(): Integer
+ resolveEvent(ArrayList<Player>): void
+ printCard(): void
+ print(Printer): String[]
+ getPaEveMultiplierPP(): Integer

PaintingEventTest

~ p1: Player
~ visitor: CharacterVisitor
~ p2: Player

~ paintingEventGettersTest(): void
+ printCardTest(): void
~ paintingEventTest(): void

<<record>>
Pick

- username: String
- index: int
- skip: boolean
- isUpper: boolean
- round: int
- isBuilding: boolean

+ isUpper(): boolean
+ round(): int
+ skip(): boolean
+ index(): int
+ isBuilding(): boolean
+ username(): String

PickCommand

- pick: Pick

+ execute(VirtualClientInterface, ServerController): void

PickCommand

- second: boolean
- third: int
~ success: boolean
- first: boolean

- parseBoolean(String): boolean
+ execute(ClientController, TUI): void

PickError

+ execute(Socket, ClientController): void

PickedCardCommand

~ p: Pick

+ execute(Socket, ClientController): void

Picker

- discount: int

+ print(Printer): String[]
+ addCard(CharacterVisitor, Player): boolean
+ getDiscount(): int
+ findBox(GuiVisitor): VBox
+ getImagePath(): String

PickerTest

~ getDiscountTest(): void
~ getDiscountShouldReturnMinusThree(): void
~ painterImagePathTest(): void
~ PickerPrintTest(): void

PingCommand

+ execute(Socket, ClientController): void

PingCommandServer

+ execute(VirtualClientInterface, ServerController): void

PingManager

- scheduler: ScheduledExecutorService
~ clients: ArrayList<VirtualClientInterface>
- delay: int

+ close(): void
- sendPingToClients(): void
+ startPingTask(): void
+ addClientToPingList(VirtualClientInterface): void

PingManagerClientSide

- scheduler: ScheduledExecutorService

+ close(): void
+ start(ServerConnection): void

PingManagerTest

- pingManager: PingManager
- client: Client

~ tearDown(): void
~ testPingFailureTriggersExceptionHandling(): void
+ addClientToPingListTest(): void
~ closeTest(): void
~ testStartPingTask(): void
~ setUp(): void

Player

- hunterCounter: int
- builderCounter: int
- serialVersionUID: long
- food: AtomicInteger
- pickerCounter: int
- builderDiscount: int
- name: String
- shamanCounter: int
- painterCounter: int
- virtualClient: VirtualClientInterface
- tribeCard: ArrayList<Character>
- buildings: ArrayList<Building>
- totem: Totem
+ countersLock: Object
- inventorCounter: int
- prestigePoints: AtomicInteger
- starCounter: int

+ getBuilderCounter(): int
+ countTribeCardsByIcon(String): int
+ modifyFood(int): void
+ finalScore(): int
+ setHunterCounter(int): void
+ getStarCounter(): int
+ modifyPP(int): void
+ painterBonus(): int
+ getPainterCounter(): int
+ builderBonus(): int
+ getBuilding(): ArrayList<Building>
+ getHunterCounter(): int
+ getInventorCounter(): int
+ inventorBonus(): int
+ getBuilderDiscount(): int
+ modifyStarCounter(int): void
+ getTotem(): Totem
+ setPickerCounter(int): void
+ increaseBuilderDiscount(int): void
+ countSet(): int
+ getFood(): int
+ getTribeCard(): ArrayList<Character>
+ getName(): String
+ setBuilderCounter(int): void
+ getPickerCounter(): int
+ getPrestigePoints(): int
+ setInventorCounter(int): void
+ getShamanCounter(): int
+ setShamanCounter(int): void
+ buildingMultipliedBonus(): int
+ getVirtualClient(): VirtualClientInterface
+ buildingBonus(): int
+ setPainterCounter(int): void

PlayerHandView

- player: Player
- boxes: Map<Class, VBox>
- building: HBox
- hand: HBox

+ getBuilding(): HBox
+ fillPlayerHand(): void
+ getHand(): HBox

<<record>>
PlayerScore

- points: int
- food: int
- username: String

+ points(): int
+ food(): int
+ username(): String

PlayerTest

+ PlayerFoodModifyTest(): void
~ buildingMultipliedBonus(): void
~ buildingBonusShouldSumPPOfAllBuildings(): void
+ BuilderBonusPlayerTest(): void
+ settersAndGettersCounterTest(): void
+ painterBonusPlayerTest(): void
+ countSetPlayerTest(): void
+ buildingMultipliedBonusShouldUseMultiplicationBuilding(): void
~ buildingMultipliedBonustest(): void
+ PlayerConstructorTest(): void
+ countTribeCardByIconTest(): void
+ buildingBonusShouldSumBuildingPP(): void
+ buildingMultipliedBonusShouldReturnZeroWithoutMultiplicationBuilding
~ buildingBonusCoverageTest(): void
+ InventorBonusPlayerTest(): void
~ buildingMultipliedBonusTest(): void
~ buildingBonusTest(): void
+ PlayerPrestigePointsModifyTest(): void
+ finalScoreTest(): void

PlayerTurnCommand

- username: String

+ execute(Socket, ClientController): void

Playerpanel

- foodLabel: Label
- PPLabel: Label
- guiView: GuiView
- playerHandView: PlayerHandView
- player: Player

+ setPlayer(Player): void
+ refreshFood(Player): void
+ openPlayerHand(Player): void
+ refreshPP(Player): void
+ getPlayerHandView(): PlayerHandView

Printer

- top: char
~ msgStyle: AttributedStyle
- side: char
- bottomRightCorner: char
~ redStyle: AttributedStyle
~ whiteStyle: AttributedStyle
~ blackStyle: AttributedStyle
~ errStyle: AttributedStyle
- topLeftCorner: char
- topRightCorner: char
- bottomLeftCorner: char
~ communicationsStyle: AttributedStyle
- bottom: char
- downArrow: char
~ blueStyle: AttributedStyle
- height: int
~ yellowStyle: AttributedStyle
- space: char
- upArrow: char

+ print(AddCard): String[]
+ print(HuntingEvent): String[]
+ print(DoubleBonusBuilding): String[]
+ print(BonusFood): String[]
+ print(NoMalusBuilding): String[]
+ print(Shaman): String[]
+ print(MultiplicationBuilding): String[]
+ print(DiscountBuilding): String[]
+ print(SameIconBuilding): String[]
+ print(Painter): String[]
+ print(Inventor): String[]
+ print(SustenanceEvent): String[]
+ encapsulator(String[]): String[]
+ print(Hunter): String[]
+ print(Picker): String[]
+ print(PaintingEvent): String[]
+ print(BonusStarBuilding): String[]
+ print(TurnOrderCard): String[]
+ print(ShamanicEvent): String[]
+ print(Builder): String[]
+ initialize(String[]): String[]
+ print(MultiplierPPBuilderBuilding): String[]
+ print(BonusPPBuilding): String[]
+ print(SetBonus): String[]
+ printTOC(Totem[]): String[]
+ print(OfferCard): String[]

PrintingHandler

- terminal: Terminal
- pipeline: BlockingQueue<String>
- lock: AtomicBoolean
- writer: PrintWriter
- reader: LineReader
- printerExec: ExecutorService

+ close(): void
+ screenCleaner(): void
+ unlockPipeline(): void
+ lockPipeline(): void
+ run(): void
+ addToPipeline(String): void

QuitCommand

+ execute(VirtualClientInterface, ServerController): void

RMIVirtualClient

- client: RemoteClientInterface

+ numPlayersError(): void
+ updateStartGame(ArrayList<Player>, Board): void
+ totemChoiceError(): void
+ ping(): void
+ updateForEvent(Event): void
+ returnTotemToTOC(String, int): void
+ showPlayerTurn(String): void
+ updateNextRound(Board, int): void
+ usernameError(): void
+ showUpdateEra(int): void
+ updateConfirmedTotem(Totem): void
+ updateLeaderboardFromDB(int, List<LeaderBoardData>): void
+ pickCardError(): void
+ updateEndGame(String, List<PlayerScore>): void
+ skipError(): void
+ skipTurn(): void
+ refuseConnection(): void
+ updateForcedEndGame(): void
+ pickedCard(String, boolean, boolean, int, int): void
+ movedTotem(String, int): void
+ updatePlayerFood(String, int): void
+ showChosenNumPlayers(int): void
+ updateFirstPlayer(): void
+ movedTotemError(): void
+ updateAvailableTotems(ArrayList<Totem>): void
+ updateConfirmedUsername(String): void
+ updatePlayerPP(String, int): void

RefusedConnectionCommand

+ execute(Socket, ClientController): void

<<interface>>
RemoteClientInterface

+ showUpdateForEvent(Event): void
+ refuseConnection(): void
+ showChosenNumPlayers(int): void
+ updateNextTurn(Board, int): void
+ numPlayersChosenError(String): void
+ showLeaderboardFromDB(int, List<LeaderBoardData>): void
+ showTotemChoiceError(String): void
+ showConfirmTotem(Totem): void
+ showLocalReturnToTOC(String, int): void
+ showGameStarted(ArrayList<Player>, Board): void
+ showTotemMoved(String, int): void
+ showForcedEndGame(): void
+ showUsernameError(String): void
+ showEndGameSuccessfully(String, List<PlayerScore>): void
+ showConfirmUsername(String): void
+ showLocalUpdateEra(int): void
+ showUpdatePlayerPP(String, int): void
+ showSkipError(String): void
+ showPickCardError(String): void
+ ping(): void
+ showTotemMovedError(String): void
+ showUpdateFirstPlayer(): void
+ showUpdatePlayerFood(String, int): void
+ showPlayerTurn(String): void
+ showPickedCard(String, boolean, boolean, int, int): void
+ showAvailableTotems(ArrayList<Totem>): void

ResolvingEventCommand

~ e: Event

+ execute(Socket, ClientController): void

SameIconBuilding

~ checkPair: Map<String, Integer>

+ getCheckPair(): Map<String, Integer>
+ acceptSameIconBonus(SetAndIconVisitor, Player, String): void
+ acceptActivation(ActivationVisitor, Player): void
+ giveFoodBonus(Player, String): boolean
+ addInventorIconToMap(String): void
+ print(Printer): String[]

SameIconBuildingTest

~ factory: ConcreteFactoryEra
~ b: SameIconBuilding
~ v: ActivationVisitor
~ p: Player

~ constructorTest(): void
~ acceptTest(): void
~ addInventorIconExceptionTest(): void
~ giveFoodBonusTestNotwell(): void
~ giveFoodBonusTestGreat(): void
~ addInventorIconTest(): void
~ acceptSameIconBonusVisitorTest(): void
~ giveFoodAtSecondSameIcon(): void
~ printTest(): void
~ acceptActivationAndVisitorTest(): void

SelectTotemCommand

- totem: String

+ execute(ClientController, TUI): void

Server

- serverSocket: ServerSocket
~ isTerminationStarted: AtomicBoolean
~ serverController: ServerController
- serverRMI: ServerRMI

+ main(String[]): void
+ closeConnection(VirtualClientInterface): void
+ terminate(): void

<<interface>>
ServerCommand

+ execute(VirtualClientInterface, ServerController): void

<<interface>>
ServerConnection

+ close(): void
+ requestAvailableTotems(): void
+ ping(): void
+ requestMoveTotem(String, int): void
+ login(String, Totem): void
+ requestPickCard(String, boolean, boolean, int, boolean): void
+ requestSetNumPlayers(int): void
+ leave(): void

ServerController

- gameManager: GameManager
- lobby: Lobby
- pingManager: PingManager
- notifier: Notifier

+ checkStartGame(): void
+ closeConnections(VirtualClientInterface): void
+ getLobby(): Lobby
+ getNotifier(): Notifier
+ checkUsername(String, VirtualClientInterface): void
+ pushPlayersInGM(ArrayList<TempPlayer>): void
+ gameInitializer(int, ArrayList<TempPlayer>): void
+ moveTotemRequest(String, int): void
+ addPlayerToGame(String, Totem, VirtualClientInterface): void
+ genericPick(String, boolean, boolean, int, boolean): void
+ setNotifier(Notifier): void
+ connectionInitializer(VirtualClientInterface): void
+ getGM(): GameManager
+ handleDisconnection(VirtualClientInterface): void
+ getPingManager(): PingManager
+ checkSetNumPlayers(int, VirtualClientInterface): void
+ checkTotem(Totem, VirtualClientInterface): void

ServerControllerTest

- gm: GameManagerFake
- client5: Client
- client2: Client
- serverController: ServerController
- client4: Client
- client3: Client
- notifier: Notifier
- pingManager: PingManager
- client1: Client

+ checkStartGameNotSuccess_Test(): void
+ gameInizialized_Test(): void
+ setterTest(): void
~ testCheckUsername_SameUserResubmitsSameName(): void
+ checkStartGamePlayersNotReady_Test(): void
+ checkTotemSamePlayerReselectsSame_Test(): void
~ gettersTest(): void
+ checkStartGame_InvalidRange_Test(): void
+ checkStartGameSuccess_Test(): void
+ checkSetNumPlayersSuccess_Test(): void
+ pushPlayersInGM_DoesNothing_WhenPlayersAlreadyExist_Test(): void
+ CheckSetNumPlayers_InvalidRange_Test(): void
+ pushPlayersInGM_Test(): void
+ checkUsername_Test(): void
+ setup(): void
~ checkUsername_DuplicateName_Test(): void
+ checkTotem_RefuseConnectionWhenNoTotemsAvailable_Test(): void
+ checkTotem_Test(): void
+ checkTotemDuplicate_Test(): void
+ genericPick_ShouldCallSkipPick_WhenSkipIsTrue_Test(): void
+ genericPick_ShouldCallResolvePick_WhenSkipIsFalse_Test(): void
+ checkSetNumPlayers_UnauthorizedUser_Test(): void
+ moveTotemRequest_Test(): void

ServerRMI

- gameManager: GameManager
- activeClients: Map<RemoteClientInterface, VirtualClientInterface>
- serverController: ServerController
- lobby: Lobby

+ requestPickCardManagement(String, boolean, boolean, int, boolean): voi
+ login(String, Totem, RemoteClientInterface): void
+ requestSetNumPlayersManagement(int, RemoteClientInterface): void
+ leave(RemoteClientInterface): void
+ requestMoveTotemManagement(String, int): void
+ connect(RemoteClientInterface): void
+ requestAvailableTotemsManagement(RemoteClientInterface): void
+ ping(): void
+ closeRMI(): void
+ run(): void

ServerSocket

- lobby: Lobby
+ termination: boolean
+ serverSocket: ServerSocket
- serverController: ServerController
- gameManager: GameManager
+ clientHandlers: ArrayList<ClientHandler>

+ run(): void
+ close(): void

<<interface>>
SetAndIconVisitor

+ visit(BonusPPBuilding, Player, String): void
+ visit(DoubleBonusBuilding, Player, String): void
+ visit(DiscountBuilding, Player, String): void
+ visit(SetBonus, Player, String): void
+ visit(BonusFood, Player, String): void
+ visit(MultiplierPPBuilderBuilding, Player, String): void
+ visit(AddCard, Player, String): void
+ visit(BonusStarBuilding, Player, String): void
+ visit(MultiplicationBuilding, Player, String): void
+ visit(SameIconBuilding, Player, String): void
+ visit(NoMalusBuilding, Player, String): void

SetBonus

- fullSetCounter: int

+ acceptActivation(ActivationVisitor, Player): void
+ print(Printer): String[]
+ giveExtraFoodSet(Player): boolean
+ getFullSetCounter(): int
+ acceptSetBonus(SetAndIconVisitor, Player): void
+ setFullSetCounter(int): void

SetBonusTest

~ factory: ConcreteFactoryEra
~ v: ActivationVisitor
~ p1: Player
~ b: SameIconBuilding

~ constructorTest(): void
~ giveFoodWhenSetIsCompleted(): void
~ printTest(): void
~ giveNoFoodWhenSetIsNotCompleted(): void
~ acceptActivationVisitorTest(): void
~ setterMethodTest(): void

SetNumPlayersCommand

- numPlayers: int

+ execute(VirtualClientInterface, ServerController): void

SetNumPlayersError

+ execute(Socket, ClientController): void

SetNumPlayersRequestCommand

~ numPlayers: String

+ execute(ClientController, TUI): void

SetTotemCommand

~ totem: Totem

+ execute(VirtualClientInterface, ServerController): void

SetUsernameCommand

~ username: String

+ execute(VirtualClientInterface, ServerController): void

SetUsernameCommand

- username: String

+ execute(ClientController, TUI): void

Shaman

- shamanStars: Integer

+ print(Printer): String[]
+ addCard(CharacterVisitor, Player): boolean
+ getImagePath(): String
+ findBox(GuiVisitor): VBox
+ getShamanStars(): Integer

ShamanTest

~ shamanPrintTest(): void
~ shamanImagePathTest(): void

ShamanicAbstractVisitor

+ visit(DoubleBonusBuilding, Player, ShamanicEvent): void
+ visit(SetBonus, Player, ShamanicEvent): void
+ visit(AddCard, Player, ShamanicEvent): void
+ visit(BonusStarBuilding, Player, ShamanicEvent): void
+ visit(DiscountBuilding, Player, ShamanicEvent): void
+ visit(MultiplierPPBuilderBuilding, Player, ShamanicEvent): void
+ visit(NoMalusBuilding, Player, ShamanicEvent): void
+ visit(BonusFood, Player, ShamanicEvent): void
+ visit(BonusPPBuilding, Player, ShamanicEvent): void
+ visit(MultiplicationBuilding, Player, ShamanicEvent): void
+ visit(SameIconBuilding, Player, ShamanicEvent): void

ShamanicDoubleBonusVisitor

+ visit(DoubleBonusBuilding, Player, ShamanicEvent): void

ShamanicDoubleBonusVisitorTest

+ VisitModifiesPlayerPPCorrectlyTest(): void

ShamanicEvent

- MaxStars: Integer
- MinStars: Integer
- ShEvePenPoints: Integer
- ShEvePrizePoints: Integer

+ getShEvePenPoints(): Integer
+ getMinStars(ArrayList<Player>): int
+ getShEvePrizePoints(): Integer
+ getMaxStars(ArrayList<Player>): int
+ printCard(): void
+ resolveEvent(ArrayList<Player>): void
+ print(Printer): String[]

ShamanicEventTest

~ onlyPlayer(): void
+ printCardTest(): void
~ playersWithMoreStars(): void
~ resolveEventShamanic(): void

ShamanicNoMalusVisitor

+ visit(NoMalusBuilding, Player, ShamanicEvent): void

<<interface>>
ShamanicVisitorInterface

+ visit(BonusStarBuilding, Player, ShamanicEvent): void
+ visit(BonusPPBuilding, Player, ShamanicEvent): void
+ visit(MultiplierPPBuilderBuilding, Player, ShamanicEvent): void
+ visit(SameIconBuilding, Player, ShamanicEvent): void
+ visit(DoubleBonusBuilding, Player, ShamanicEvent): void
+ visit(MultiplicationBuilding, Player, ShamanicEvent): void
+ visit(AddCard, Player, ShamanicEvent): void
+ visit(NoMalusBuilding, Player, ShamanicEvent): void
+ visit(SetBonus, Player, ShamanicEvent): void
+ visit(BonusFood, Player, ShamanicEvent): void
+ visit(DiscountBuilding, Player, ShamanicEvent): void

ShowAvailableColorsCommand

~ totems: ArrayList<Totem>

+ execute(Socket, ClientController): void

ShowCardsCommand

~ totem: String

+ execute(ClientController, TUI): void

ShowMSGCommand

- message: String

+ execute(Socket, ClientController): void

SkipCommand

+ execute(Socket, ClientController): void

SkipCommand

+ execute(ClientController, TUI): void

SkipErrorCommand

+ execute(Socket, ClientController): void

SocketClient

- socketPort: int
- in: ObjectInputStream
- pool: ExecutorService
- closeSignal: boolean
+ clientController: ClientController
+ out: ObjectOutputStream
- serverIP: String
- socket: Socket
+ outputLock: Object

+ login(String, Totem): void
+ send(String, Object): void
+ leave(): void
+ requestAvailableTotems(): void
+ requestSetNumPlayers(int): void
+ close(): void
+ requestMoveTotem(String, int): void
+ run(): void
+ sendTotem(Totem): void
+ requestPickCard(String, boolean, boolean, int, boolean): void
+ ping(): void

SocketVirtualClient

- out: ObjectOutputStream
- socket: Socket
- outputLock: Object
- gm: GameManager

+ pickCardError(): void
+ ping(): void
+ updateFirstPlayer(): void
+ movedTotem(String, int): void
+ updateAvailableTotems(ArrayList<Totem>): void
+ updateStartGame(ArrayList<Player>, Board): void
+ showPlayerTurn(String): void
+ movedTotemError(): void
+ refuseConnection(): void
+ updatePlayerPP(String, int): void
+ usernameError(): void
+ totemChoiceError(): void
+ updateConfirmedUsername(String): void
+ updateForEvent(Event): void
+ pickedCard(String, boolean, boolean, int, int): void
+ updateEndGame(String, List<PlayerScore>): void
+ updateForcedEndGame(): void
+ numPlayersError(): void
+ returnTotemToTOC(String, int): void
+ showUpdateEra(int): void
+ updatePlayerFood(String, int): void
+ confirmNumPlayers(int): void
+ updateNextRound(Board, int): void
- send(String, T): void
+ showChosenNumPlayers(int): void
+ updateLeaderboardFromDB(int, List<LeaderBoardData>): void
+ skipTurn(): void
+ skipError(): void
+ updateConfirmedTotem(Totem): void

StartGameCommand

~ gameStartData: GameStartData

+ execute(Socket, ClientController): void

SustenanceEvent

- SuEvePointsLossMultiplier: Integer

+ getSuEvePointsLossMultiplier(): Integer
+ print(Printer): String[]
+ printCard(): void
+ resolveEvent(ArrayList<Player>): void

SustenanceEventTest

~ p2: Player
~ p1: Player

~ payCharacterFoodInTheTribe_Test(): void
~ nameShouldReturnCorrectString(): void
+ payCharacterInTheTribeWithFoodAndPP_Test(): void
~ constructorTest(): void
~ sustenancePrintCardDoesNotThrow(): void
~ valueOfCorrectEventName(): void
+ payCharacterInTheTribeWithPP_Test(): void

TUI

- printExecutor: ExecutorService
~ invalidConnectionInput: String
- controller: ClientController
- writer: PrintWriter
- turnTotem: Totem
~ redStyle: AttributedStyle
- printingHandler: PrintingHandler
- currentPlayer: String
~ yellowStyle: AttributedStyle
~ connectingToServer: String
- turnLock: Object
- LOCK: Object
- numPlayersPopUp: AtomicBoolean
- cardsPopUp: AtomicBoolean
~ blueStyle: AttributedStyle
~ blackStyle: AttributedStyle
~ connectionChoice: String
- sc: Scanner
- totemStylesMap: Map<Totem, AttributedStyle>
~ whiteStyle: AttributedStyle
~ errStyle: AttributedStyle
- isRunning: AtomicBoolean
- terminal: Terminal
- tocLock: Object
~ invalidConnectionChoice: String
- reader: LineReader
~ msgStyle: AttributedStyle
- TOC: Totem[]
- printer: Printer
~ writeHere: AttributedString
~ communicationsStyle: AttributedStyle

+ printCardsRow(ArrayList<Card>): String[]
+ showEndGameSuccessfully(String, List<PlayerScore>): void
+ startMenu(): void
+ boardPrinter(Board, String): void
- initializeColorsMap(): void
+ showNextRound(): void
+ close(): void
+ showLeaderboardFromDB(int, List<LeaderBoardData>): void
+ showCurrentPlayer(String): void
- removeToTOC(Totem): void
+ printPath(TurnOrderCard, ArrayList<OfferCard>): String[]
+ showLobbyMenu(): void
+ showError(String): void
- menuFormatter(AttributedStringBuilder, String, String, String): void
- handleNumPlayersInput(String): void
- addInHead(Totem): void
+ run(): void
+ showPlayerPPUpdate(String, int): void
+ printBuildingRow(ArrayList<Card>): String[]
+ showStartGame(): void
+ printPlayerCards(Player): void
+ showPlayerFoodUpdate(String, int): void
+ pickCard(String, boolean, boolean, int): void
- handleCardsRequest(String): void
+ cardMerger(String[]): String[]
+ initializeTOC(): void
+ printPlayers(ArrayList<Player>): String[]
+ showAvailableTotems(ArrayList<Totem>): void
+ showCardsCommandTui(String): void
+ showReturnToTOC(String): void
+ moveTotem(String, int): void
+ showMessage(String): void
+ showErrorMessage(String): void
+ askNumToPlayer(): void
- resetTOC(): void
+ screenCleaner(): void

TempPlayer

- name: String
- totem: Totem
- client: VirtualClientInterface

+ setName(String): void
+ getName(): String
+ getTempPlayerTotem(): Totem
+ getClient(): VirtualClientInterface
+ setTempPlayerTotem(Totem): void

TempPlayerTest

~ testTempPlayerFunctionality(): void

<<enumeration>>
Totem

+ BLUE:
+ YELLOW:
+ WHITE:
+ BLACK:
+ ORANGE:

+ values(): Totem[]
+ valueOf(String): Totem

TotemError

~ totems: ArrayList<Totem>

+ execute(Socket, ClientController): void

<<record>>
TotemPosition

- index: int
- playerName: String

+ playerName(): String
+ index(): int

TotemPositionError

+ execute(Socket, ClientController): void

TurnOrderCard

- serialVersionUID: long
- NumPlayers: int
- foodContainer: Integer[]
- order: ArrayList<Player>

+ getHead(): Totem
+ getNumPlayers(): int
+ getFoodByIndex(int): int
+ print(Printer): String[]
+ getOrder(): ArrayList<Player>
- makeTwoPlayers(): Integer[]
+ initializeTOC(int): void
- makeThreePlayers(): Integer[]
- makeFourPlayers(): Integer[]
- makeFivePlayers(): Integer[]

<<interface>>
TurnOrderCardFoodBonus

+ visit(BonusStarBuilding, Player): void
+ visit(BonusFood, Player): void
+ visit(AddCard, Player): void
+ visit(MultiplierPPBuilderBuilding, Player): void
+ visit(SetBonus, Player): void
+ visit(MultiplicationBuilding, Player): void
+ visit(SameIconBuilding, Player): void
+ visit(NoMalusBuilding, Player): void
+ visit(BonusPPBuilding, Player): void
+ visit(DiscountBuilding, Player): void
+ visit(DoubleBonusBuilding, Player): void

TurnOrderCardTest

~ makeFourPlayersFoodTest(): void
~ makeTwoPlayersFoodTest(): void
~ getHeadShouldThrowIfOrderIsEmpty(): void
~ TOCConstructorTest(): void
~ getFoodIndexLargerThanLenght(): void
~ printShouldReturnPrinterResult(): void
~ getHeaRemovePlayer(): void
~ getFoodForFivePlayers(): void
~ getFoodCorrectIndex(): void
~ makeThreePlayersFoodTest(): void
~ getFoodIndexMinusNegative(): void

TurnOrderPositionCommand

~ totemPosition: TotemPosition

+ execute(Socket, ClientController): void

UnknownCommand

+ execute(ClientController, TUI): void

UnknownCommand

+ execute(VirtualClientInterface, ServerController): void

UpdateEraCommand

~ era: int

+ execute(Socket, ClientController): void

<<record>>
UpdateFood

- food: int
- playerName: String

+ playerName(): String
+ food(): int

UpdateFoodCommand

- command: UpdateFood

+ execute(Socket, ClientController): void

<<record>>
UpdatePP

- playerName: String
- update: int

+ playerName(): String
+ update(): int

UpdatePPCommand

- command: UpdatePP

+ execute(Socket, ClientController): void

UsernameErrorCommand

+ execute(Socket, ClientController): void

Utils

+ createImageView(String, double): ImageView

<<interface>>
VirtualClientInterface

+ showUpdateEra(int): void
+ totemChoiceError(): void
+ refuseConnection(): void
+ movedTotem(String, int): void
+ ping(): void
+ updateForcedEndGame(): void
+ movedTotemError(): void
+ showPlayerTurn(String): void
+ updateNextRound(Board, int): void
+ returnTotemToTOC(String, int): void
+ updatePlayerPP(String, int): void
+ updateFirstPlayer(): void
+ updateEndGame(String, List<PlayerScore>): void
+ showChosenNumPlayers(int): void
+ numPlayersError(): void
+ updateAvailableTotems(ArrayList<Totem>): void
+ skipTurn(): void
+ pickCardError(): void
+ updateLeaderboardFromDB(int, List<LeaderBoardData>): void
+ updateForEvent(Event): void
+ pickedCard(String, boolean, boolean, int, int): void
+ updatePlayerFood(String, int): void
+ updateConfirmedUsername(String): void
+ updateConfirmedTotem(Totem): void
+ usernameError(): void
+ updateStartGame(ArrayList<Player>, Board): void
+ skipError(): void

<<interface>>
VirtualServer

+ connect(RemoteClientInterface): void
+ login(String, Totem, RemoteClientInterface): void
+ leave(RemoteClientInterface): void
+ ping(): void
+ requestSetNumPlayersManagement(int, RemoteClientInterface): void
+ requestMoveTotemManagement(String, int): void
+ requestPickCardManagement(String, boolean, boolean, int, boolean): voi
+ requestAvailableTotemsManagement(RemoteClientInterface): void

<<interface>>
VirtualView

+ showMyTurn(): void
+ updateOtherPlayerStatus(String, List<Card>, List<Building>): void
+ showCurrentPlayer(String): void
+ showStartGame(String, ArrayList<Player>, Totem, int): void
+ showEndGame(int): void
+ showError(String): void
+ updateBoardStatus(Board): void

1

*

buildingsEra1

1

*

upperCardRow

1

*

path

1

* players

1

1

turnOrderCard

1

1

board

1

1

b

1

1

player1

1

1

clientController

1

1

guiView

1

1 bonusFood

1

1

player

1

1 bonusPPBuilding

1

1

player

1

1

typeEvents

1

1

typeIcons

1

* era1

1

1 characterType

1

currentBoard 1
1

1 view

1

*

leaderboardDB

1
1

pingManager

1

1 serverConnection

1

1

tmpTotem

1

1 serverController

1

1

client

1

1

clientController

1

1

tui

1

1

clientController

1

1 server

1

1 totem

1

*

leaderboard

1

*

deck

1

1 typeEvents

1

1

typeIcons

1

*

buildings

1

buildingFactory 1

1

*

characters

1

1

factory

1

*

events

* 1
players

1

1 building

1 1

player

1

1 event

1

* leaderboard

1

*

characters

1

* events

1

1

board

1

1

notifier

1

*

players

1

player1 1

1

1

boardView

1

1
clientController

1

1 menuView

1

panels *

1

1

visitor

1

1

p1

1

1

serverController

1

*

tempPlayers

1

1

lobby

1

1

clientController

1

1 clientController

1

guiView 1

1

1

totemPosition

1

1

pos

1

*

characters

1

1 factory

1

1
p

1

1 building

1

1

p1

1

board 1

1

1

v

1

1 b

1

1

p

1

*

clients

1

1

OccupiedBy

1

1

visitor

1

1
p1

1

1

pick

1

1

p

1

*

clients

1

1

pingManager

1

*

buildings

1

*

tribeCard

1

1

totem

1

1

virtualClient

1

1

player

1

1
guiView

1

1

player

1
1

playerHandView

1

client 1

1

1
e

1

1

v

1

1

factory

1

1

p

1

1 b

1

1

serverController

1

1

serverRMI

1 1

serverSocket

1

1

gameManager

1

1

lobby

1

1

notifier

1

1

pingManager

1

1

notifier

1

1
pingManager

1
1

serverController

1

1

gameManager

1

1

lobby

1

*

activeClients

1

1

serverController

1

*

activeClients

1

*

clientHandlers

1

1

gameManager

1
1

lobby

1

1

serverController

1

1

v

1

1 factory

1

1

p1

1

1 b

1

1

totem

1

*

totems

1

1
clientController

1

1

gm

1

1
p1

1

1

controller

1

1
printer

1

1

printingHandler

1

*

TOC

1

1totem

1

client 1

1

*

totems

1

* order

1

1

totemPosition

1

1 command

1

1 command